

INSTITUTO MAUÁ DE TECNOLOGIA



Lógica e Pensamento Computacional

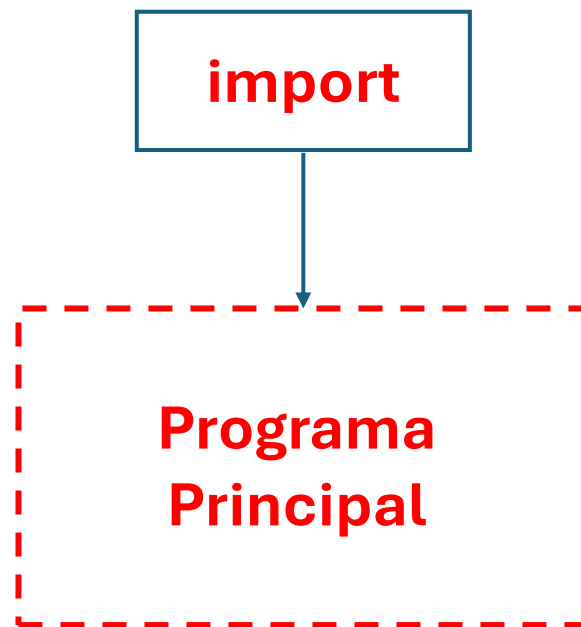
Arrays e Biblioteca Numpy

OBJETIVOS

- Revisar os conceitos das aulas anteriores
- Apresentar o conceito de arrays junto com a biblioteca numpy
- Resolver os exercícios propostos

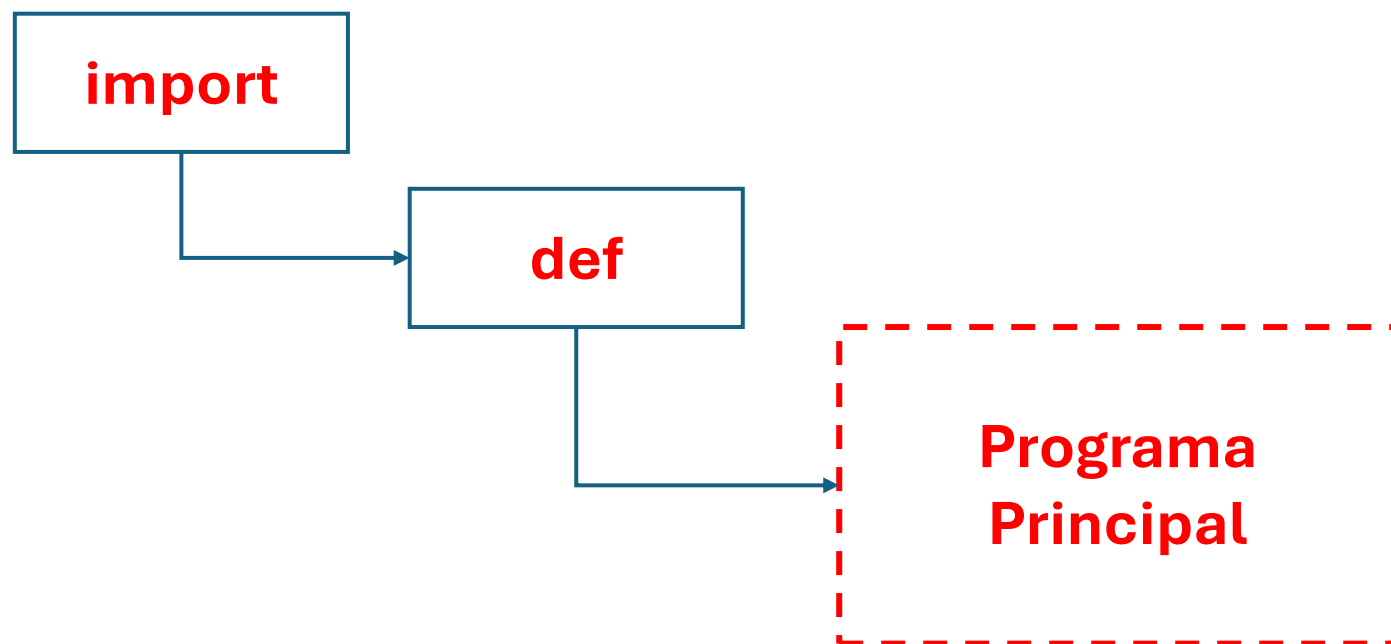
Bibliotecas: Repositório de processos já pré-programados. Normalmente as bibliotecas possuem uma temática (cálculos, gráficos, animações, sistema, etc)

Para utilizá-las no Python, vamos utilizar o comando **import**. É importante que este comando venha **antes** do programa principal, para o computador importar os comandos da biblioteca antes de utilizá-los em qualquer processo



Subrotinas: Processos pré-programados pelo próprio usuário para serem chamados em outras sub-rotinas e/ou no programa principal.

Para utilizá-las no Python, vamos utilizar o comando **def**. É importante que este comando venha **antes** do programa principal, mas **depois** do **import** das bibliotecas.



Existem dois grupos básicos de sub-rotinas:

- **Funções:** Sub-rotinas que retornam resultados para a linha onde foram “chamadas”. A função deverá ter um “nome” e os resultados deverão ser armazenados em variáveis.
- **Procedimentos:** Sub-rotinas que não retornam variáveis para a linha onde foi chamada. Normalmente são utilizados para “funções do sistema” (abrir arquivos, salvar arquivos) ou para criar rotinas de **print()** para cabeçalhos ou apresentação de resultados

Vimos que uma das vantagens das funções é a capacidade repetição da mesma sub-rotina:

```
#  
#Subrotinas  
def reta(x):  
    fx = 2*x + 3  
    return fx  
  
#  
#Programa Principal  
X1 = float(input("Digite o valor de X1: "))  
X2 = float(input("Digite o valor de X2: "))  
X3 = float(input("Digite o valor de X3: "))  
  
Resultado1 = reta(X1)  
Resultado2 = reta(X2)  
Resultado3 = reta(X3)  
  
print("O valor da função no ponto %.2f é %.2f"%(X1,Resultado1))  
print("O valor da função no ponto %.2f é %.2f"%(X2,Resultado2))  
print("O valor da função no ponto %.2f é %.2f"%(X3,Resultado3))  
  
... Digite o valor de X1: 5  
    Digite o valor de X2: 4.867  
    Digite o valor de X3: 10.12  
    O valor da função no ponto 5.00 é 13.00  
    O valor da função no ponto 4.87 é 12.73  
    O valor da função no ponto 10.12 é 23.24
```

Mas se o número de repetições for grande, o programa principal se torna grande mesmo com sub-rotinas:

Também surge o problema da quantidade de variáveis criadas

O programa principal ficou muito grande

```
#
#Subrotinas
def reta(x):
    fx = 2*x + 3
    return fx

#
#Programa Principal
X1 = float(input("Digite X1: "))
X2 = float(input("Digite X2: "))
X3 = float(input("Digite X3: "))
X4 = float(input("Digite X4: "))
X5 = float(input("Digite X5: "))
X6 = float(input("Digite X6: "))
X7 = float(input("Digite X7: "))
X8 = float(input("Digite X8: "))
X9 = float(input("Digite X9: "))
X10 = float(input("Digite X10: "))

Resultado1 = reta(X1)
Resultado2 = reta(X2)
Resultado3 = reta(X3)
Resultado4 = reta(X4)
Resultado5 = reta(X5)
Resultado6 = reta(X6)
Resultado7 = reta(X7)
Resultado8 = reta(X8)
Resultado9 = reta(X9)
Resultado10 = reta(X10)

print("O valor da função no ponto %.2f é %.2f"%(X1,Resultado1))
print("O valor da função no ponto %.2f é %.2f"%(X2,Resultado2))
print("O valor da função no ponto %.2f é %.2f"%(X3,Resultado3))
print("O valor da função no ponto %.2f é %.2f"%(X4,Resultado4))
print("O valor da função no ponto %.2f é %.2f"%(X5,Resultado5))
print("O valor da função no ponto %.2f é %.2f"%(X6,Resultado6))
print("O valor da função no ponto %.2f é %.2f"%(X7,Resultado7))
print("O valor da função no ponto %.2f é %.2f"%(X8,Resultado8))
print("O valor da função no ponto %.2f é %.2f"%(X9,Resultado9))
print("O valor da função no ponto %.2f é %.2f"%(X10,Resultado10))
```

Para resolver esse problema, o Python possui outros tipos de variáveis, cuja característica principal é armazenar uma “estrutura” de valores como se fosse uma única informação.

No curso trabalharemos duas delas:

- **Arrays:** Conjunto de valores numéricos que é utilizado junto com a biblioteca **numpy**.
- **Listas:** Conjunto de valores (numéricos ou não) que já está na configuração padrão do Python, sem a necessidade de importar bibliotecas



Aulas seguintes

NumPy significa: **N**umerical **P**ython

É uma biblioteca criada para trabalhar com **cálculos numéricos e científicos** em Python.

Ela é muito utilizada em áreas como:

- Engenharia
- Física
- Estatística
- Ciência de dados
- Inteligência Artificial

O principal recurso do NumPy é o **array numérico**, tornando-se vantajosa em relação a biblioteca **math** quando são necessárias cálculos para vários valores.

Importando o **numpy**: Utiliza-se o comando **import** como qualquer outra biblioteca.



```
# _____  
#Subrotinas  
import numpy
```

Porém, uma convenção dos usuários de Python é abreviar o nome **numpy** para diminuir o número de caracteres ao se chamar um comando desta biblioteca. Para isso, junto com o comando **import**, utiliza-se o comando **as** e coloca-se a abreviação desejada:



```
# _____  
#Subrotinas  
import numpy as np
```

Abrevia-se para np

Assim, qualquer comando da biblioteca numpy será com o prefixo **np**:

Alguns exemplos comuns a biblioteca **math**:

np.pi -> Retorna o valor de Pi com muitas casas decimais (3.14159...)

np.sqrt (array) -> Retorna a raiz quadrada do array

np.sin (array) -> Calcula o seno do array

np.cos (array) -> Calcula o cosseno do array

np.tan (array) -> Calcula a tangente do array

np.radians (array) -> Converte Graus para Radianos

Agora, vamos criar **arrays** para utilizar o **numpy**. Ao longo do curso veremos algumas formas de inicializar arrays. A primeira, e mais prática, é a inserção direta, ou seja, você digita os valores desejados no próprio código, utilizando o comando **array**:

```
import numpy as np

x = np.array([0,3,5,-2,-4,3,8,-1])
```

Ou seja, esse comando cria uma variável *x* que é um conjunto numérico com os valores apresentados dentro dos colchetes e separados por vírgulas (a separação decimal é por ponto).

Assim, nesse caso, a variável *x* possui 9 valores armazenados na estrutura

Grande vantagem do **array**: Ao se realizar qualquer cálculo com uma variável **array**, ela realiza um *BROADCASTING* (Transmissão), para todos os valores dentro da estrutura:

▶ `import numpy as np`

`x = np.array([0,3,5,-2,-4,3,8,-1])` —————> Cria-se o array

`y = 2*x` —————> Realiza-se uma operação com o array x resultando no array y

`print(y)`

... [0 6 10 -4 -8 6 16 -2]

—————> O resultado são todos os elementos do array multiplicados por 2 neste caso.

Alterando nosso exemplo da reta:



```
#
```

```
#Bibliotecas
```

```
import numpy as np
```

Biblioteca numpy abreviada para np

```
#
```

```
#Subrotinas
```

```
def reta(x):
```

```
    fx = 2*x + 3
```

```
    return fx
```

Subrotina reta

```
#
```

```
#Programa Principal
```

```
x = np.array([5,4.67,3.55,9.42,-1.75,-4.32,-9.12,8,0,-1])
```

Array x criado com inserção direta de valores

```
Resultado = reta(x)
```

Apenas 1 chamada da subrotina (Broadcasting)

```
print(Resultado)
```

print simples que mostra todos os resultados

```
... [ 13.   12.34  10.1   21.84  -0.5   -5.64 -15.24  19.    3.    1. ]
```

Outras formas de criar arrays são com comandos que criam sequências numéricas pré-defindas:

np.arange (a,b,n) -> Cria um array cujo ponto inicial é “a” e tem como limite o valor “b”. A subdivisão é dada pelo passo “n”



```
import numpy as np
```

```
x = np.arange(0,10,1)
```

```
print(x)
```

```
... [0 1 2 3 4 5 6 7 8 9]
```

Biblioteca numpy abreviada para np

Array com inicio no valor 0 com limite no valor 10. Incremento (passo) de 1 em 1

IMPORTANTE: O valor colocado como “final” no arange não é colocado no array, mesmo que o passo chegasse nele.

np.linspace(a,b,n) -> Cria um array com início no ponto “a” e término no ponto “b”. “n” será a quantidade de pontos



```
import numpy as np
```

```
x = np.linspace(0,10,10)
```

```
y = np.linspace(0,10,11)
```

```
print("x=",x)
```

```
print("y=",y)
```

Array de 0 a 10 com 10 pontos no total (incluindo 0 e 10)

Array de 0 a 10 com 11 pontos no total (incluindo 0 e 10)

```
*** x= [ 0.          1.11111111  2.22222222  3.33333333  4.44444444  5.55555556
        6.66666667  7.77777778  8.88888889 10.          ]
    y= [ 0.  1.  2.  3.  4.  5.  6.  7.  8.  9. 10.]
```

np.zeros (n) -> Cria um array com n valores (todos iguais a zero)

np.ones (n) -> Cria um array com n valores (todos iguais a um)



```
import numpy as np
```

```
x = np.zeros(5)
```

```
y = np.ones(7)
```

```
print(x)
```

```
print(y)
```

```
... [0. 0. 0. 0. 0.]  
     [1. 1. 1. 1. 1. 1. 1.]
```

Array com 5 valores valendo 0

Array com 7 valores valendo 1

Serão importantes para criarmos arrays com valores constantes!

Operações com **arrays**:

Você poderá realizar operações diversos arrays desde que possuam a mesma quantidade de valores!

```
▶ import numpy as np
```

```
x = np.linspace(0,10,10)
```

```
y = np.linspace(0,10,11)
```

```
print(x+y)
```

```
***
```

```
ValueError
```

```
Traceback (most recent call last)
```

```
/tmp/ipykernel\_429/3388055971.py in <cell line: 0>()
```

```
4 y = np.linspace(0,10,11)
```

```
5
```

```
----> 6 print(x+y)
```

```
ValueError: operands could not be broadcast together with shapes (10,) (11,)
```

Exemplo 1: Utilize a sub-rotina `reta` criada anteriormente e crie um array `x` de de 0 a 5 com 20 pontos. Calcule a função para todos esses pontos e apresente o resultado:

```
#  
#Bibliotecas  
import numpy as np  
  
#  
#Subrotinas  
def reta(x):  
    fx = 2*x + 3  
    return fx  
  
#  
#Programa Principal  
  
x = np.linspace(0,5,20)  
y = reta(x)  
  
print(y)
```

```
... [ 3.          3.52631579  4.05263158  4.57894737  5.10526316  5.63157895  
      6.15789474  6.68421053  7.21052632  7.73684211  8.26315789  8.78947368  
      9.31578947  9.84210526 10.36842105 10.89473684 11.42105263 11.94736842  
     12.47368421 13.          ]
```

Exemplo 2: Utilize a sub-rotina “pop_final” da aula passada e faça 5 projeções de uma população com 30000 habitantes: Taxas de crescimento de 1,2,3,4 e 5% ao ano para 10 anos de simulação. Escreva o resultado sem casas decimais.

Dica1: Para arredondar o array, utilize o comando **np.round (array , n)**, onde o array será arredondado com n casas decimais.

Dica2: O comando exp existe no numpy. Você pode alterar o **math.exp()** para **np.exp()**

```
#
#Bibliotecas
import numpy as np

#
#Subrotinas
def pop_final(Po,r,t):
    Pt = Po*np.exp(r*t/100)
    return Pt

#
#Programa Principal

Po = 30000*np.ones(5)
r = np.array([1,2,3,4,5])
t = 10*np.ones(5)

Pt = pop_final(Po,r,t)

print("P = ",np.round(Pt,0))
```

```
... P = [33155. 36642. 40496. 44755. 49462.]
```